

4_DICIONÁRIOS {} (respostas)

September 15, 2026

1 DICIONÁRIO USANDO PYTHON

1.1 Criar dicionário sem dados

```
[57]: dicionario = {}
```

1.2 Criar dicionários com dados

Esse seria um exemplo de dicionário mais simples possível onde a chave é uma string e o valor é um número.

```
[58]: papelaria = {"Caneta": 50,  
                  "Caderno": 30,  
                  "Borracha": 20}
```

É possível criar o mesmo dicionário porém de um em um, imagine um cenário onde a cada instante um novo aluno é cadastrado.

```
[59]: notas = {}  
notas['Maria'] = 2.7  
notas['Pedro'] = 1.9  
notas['José'] = 7.3
```

1.3 Acessar itens do dicionários usando a chave

Ao invés de acessar usando o index da posição desejada como ocorre nas listas, usamos a chave.

```
[60]: print(papelaria['Caneta'])
```

50

```
[61]: print(notas['Pedro'])
```

1.9

1.4 Alterar itens do dicionário

```
[62]: papelaria['Caneta'] = 25
```

```
[63]: notas['Pedro'] = 5
```

```
[64]: print(papelaria)

{'Caneta': 25, 'Caderno': 30, 'Borracha': 20}
```

```
[65]: print(notas)

{'Maria': 2.7, 'Pedro': 5, 'José': 7.3}
```

1.5 Inserir itens do dicionário

```
[66]: papelaria['Lapiseira'] = 1
```

```
[67]: notas['Jessica'] = 8
```

```
[68]: print(papelaria)

{'Caneta': 25, 'Caderno': 30, 'Borracha': 20, 'Lapiseira': 1}
```

```
[69]: print(notas)

{'Maria': 2.7, 'Pedro': 5, 'José': 7.3, 'Jessica': 8}
```

1.6 Deletar itens do dicionário

```
[70]: del papelaria['Lapiseira']
```

```
[71]: print(papelaria)

{'Caneta': 25, 'Caderno': 30, 'Borracha': 20}
```

1.7 Acessar as chaves(keys) e os valores(values)

O dicionário em python disponibiliza duas funções que retornam as chaves e os valores existentes no dicionário.

```
[72]: papelaria.keys()
```

```
[72]: dict_keys(['Caneta', 'Caderno', 'Borracha'])
```

```
[73]: papelaria.values()
```

```
[73]: dict_values([25, 30, 20])
```

1.8 Acessar os itens do dicionário através de um loop

Existem também uma função chamada items que retorna uma chave e um valor por vez, o qual pode ser utilizado para iterarmos sobre cada par chave-valor do dicionário.

```
[74]: for key, value in papelaria.items():
      print("Chave:", key, " Valor:", value)
```

Chave: Caneta Valor: 25
Chave: Caderno Valor: 30
Chave: Borracha Valor: 20

1.9 Verificar a existência de um aluno

Essa checagem se um dado existe no dicionário é feito sempre nas chaves do dicionário e não nos valores. Isso ocorre porque a busca padrão em um dict é feita no conjunto de chaves.

```
[75]: if "Pedro" in papelaria:  
        print("Nome existente!")  
    else:  
        print("Nome inexistente!")
```

Nome inexistente!

Se você quiser verificar se um valor está presente no dicionário (e não uma chave), precisaria usar:

```
[76]: if 50 in papelaria.values():  
        print("Valor está presente!")  
    else:  
        print("Valor não existente!")
```

Valor não existente!

1.10 Dicionários com Listas e outros Dicionários

```
[77]: turmas = {  
        "Turma A": ["João", "Maria"],  
        "Turma B": ["Ana", "Pedro"]  
    }
```

```
[78]: # Para acessar o segundo aluno da Turma A podemos fazer o seguinte:  
# Ao indexar "Turma A", recebemos como retorno uma lista.  
# Se queremos acessar o segundo elemento, usamos o índice 1,  
# já que o índice do primeiro elemento é 0.  
turmas["Turma A"][1]
```

[78]: 'Maria'

Podemos também ter um dicionário onde a chave é uma string e o valor é outro dicionário.

```
[79]: alunos = {  
        "Pedro": {"Turma": "2TI-a", "Notas": [8.3, 9.1, 9.5]},  
        "Verena": {"Turma": "2TI-b", "Notas": [9.3, 9.8, 9.9]}  
    }
```

```
[80]: # Nessa nova estrutura, para acessar as notas de Pedro, por exemplo,  
# devemos usar dois pares de colchetes. O resultado será uma lista de notas.  
alunos["Pedro"]["Notas"]
```

[80]: [8.3, 9.1, 9.5]

O que vimos até aqui foi como acessar essas estruturas mistas. Agora vem uma pergunta importante: **como adicionar e remover informações?**

A resposta depende de qual é o tipo da estrutura que estamos alterando naquele momento:

- se estamos alterando uma **lista**, podemos usar `append()` e `remove()`;
- se estamos alterando um **dicionário**, adicionamos uma nova chave e podemos remover com `del`.

Vamos ver isso nas mesmas estruturas que apareceram acima.

1.10.1 1. Dicionário em que os valores são listas

Neste exemplo, `turmas` é um dicionário. Por isso, não faz sentido escrever `turmas.append(...)`.

Mas cada turma, como `turmas["Turma A"]`, é uma lista. Então podemos usar `append()` nessa lista específica.

```
[ ]: import pprint

turmas = {
    "Turma A": ["João", "Maria"],
    "Turma B": ["Ana", "Pedro"]
}

# Adicionando um aluno em uma lista que está dentro do dicionário
turmas["Turma A"].append("Jeryel")

# Removendo um aluno de uma lista que está dentro do dicionário
turmas["Turma B"].remove("Pedro")

# Adicionando uma nova chave no dicionário
turmas["Turma C"] = ["Lia", "Rafa"]

pprint.pprint(turmas)
```

1.10.2 2. Dicionário em que os valores são outros dicionários

Agora a estrutura principal `alunos` é um dicionário. Cada chave é o nome de um aluno, e cada valor é outro dicionário com `turma` e `notas`.

Para adicionar um novo aluno, criamos uma nova chave. Para remover um aluno, podemos usar `del`. E, se quisermos adicionar uma nota, aí sim usamos `append()`, porque `Notas` é uma lista.

```
[ ]: alunos = {
    "Pedro": {"Turma": "2TI-a", "Notas": [8.3, 9.1, 9.5]},
    "Verena": {"Turma": "2TI-b", "Notas": [9.3, 9.8, 9.9]}
}
```

```

# Adicionando um novo aluno no dicionário
alunos["Jeryel"] = {"Turma": "2TI-a", "Notas": [6, 7, 8]}

# Adicionando uma nova nota na lista de notas de Pedro
alunos["Pedro"]["Notas"].append(10)

# Removendo uma aluna do dicionário
del alunos["Verena"]

pprint.pprint(alunos)

```

1.10.3 3. Lista contendo dicionários

Também podemos guardar cada aluno como um dicionário dentro de uma lista. Nesse caso, a estrutura principal é uma lista, então `append()` funciona para adicionar um novo item na lista.

Atenção: essa estrutura é possível, mas para buscar um aluno pelo nome ela costuma ser menos direta do que o dicionário anterior.

```

[ ]: alunos = [
    {"Pedro": {"Turma": "2TI-a", "Notas": [8.3, 9.1, 9.5]}},
    {"Verena": {"Turma": "2TI-b", "Notas": [9.3, 9.8, 9.9]}}
]

novo_aluno = {"Jeryel": {"Turma": "2TI-a", "Notas": [6, 7, 8]}}

# Adicionando um novo dicionário na lista
alunos.append(novo_aluno)

# Removendo o primeiro dicionário da lista
aluno_removido = alunos.pop(0)

pprint.pprint(alunos)
print("Aluno removido:")
pprint.pprint(aluno_removido)

```

Resumo da ideia:

Estrutura	Como adicionar	Como remover
Lista	<code>append()</code>	<code>remove()</code> ou <code>pop()</code>
Dicionário	<code>dicionario[nova_chave] = valor</code>	<code>del dicionario[chave]</code>
Lista dentro de dicionário	<code>dicionario[chave].append(valor)</code>	<code>dicionario[chave].remove(valor)</code>

1.11 Recapitulando: lista x dicionário

Uma **lista** guarda vários valores em sequência. Para acessar um item, usamos a posição dele, chamada de índice. O primeiro item fica na posição 0, o segundo na posição 1, e assim por diante.

```
nomes = ["Pedro", "Verena", "Jeryel"]
nomes[0]
```

Um **dicionário** guarda informações em pares de **chave** e **valor**. Para acessar um item, usamos a chave, e não a posição.

```
aluno = {"Nome": "Pedro", "Turma": "2TI-a"}
aluno["Turma"]
```

Por isso, `append()` é uma função de lista. Não usamos `append()` diretamente em um dicionário.

```
# Lista: posso usar append
nomes.append("Ana")
```

```
# Dicionário: adiciono uma nova chave
aluno["Nota"] = 9.5
```

Atenção: quando existe uma lista dentro de um dicionário, podemos usar `append()` nessa lista interna.

```
alunos = {
    "Pedro": {"Notas": [8.3, 9.1]}
}
```

```
alunos["Pedro"]["Notas"].append(10)
```

Nesse caso, o `append()` não está sendo usado no dicionário `alunos`. Ele está sendo usado na lista que está guardada dentro da chave "Notas".